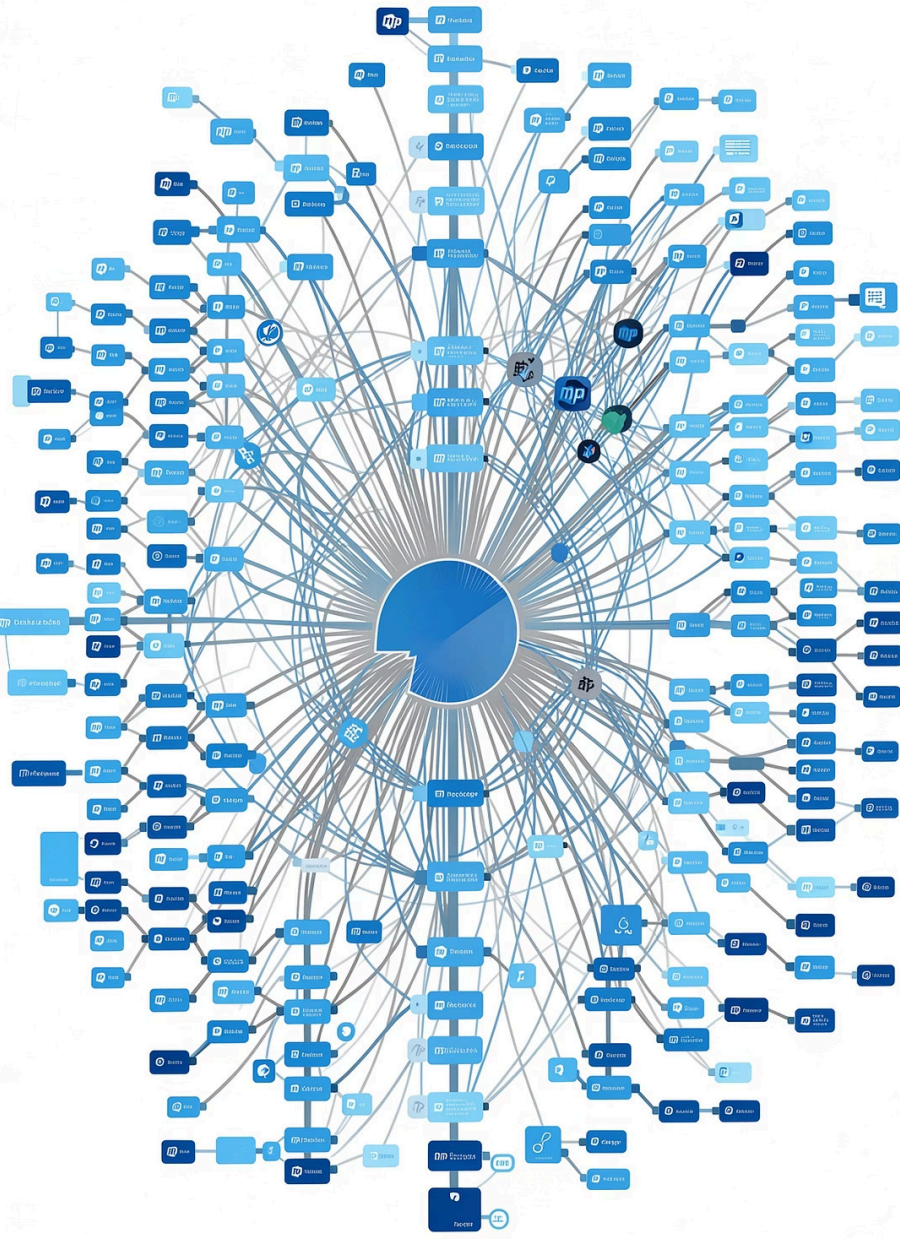
Supply Chain Attack Detection via npm Dependency Graph Analysis

MSML606 Bonus Project 2 · Spring 2026

HELEN LI · YAXITA AMIN

UNIVERSITY OF MARYLAND



You Install One Package. You Get Thousands.

The Hidden Depth Problem

A typical JavaScript project pulls in **1,000–5,000+ transitive dependencies** from just a handful of direct installs. A malicious package buried 4–5 hops deep is **completely invisible** to manual inspection.

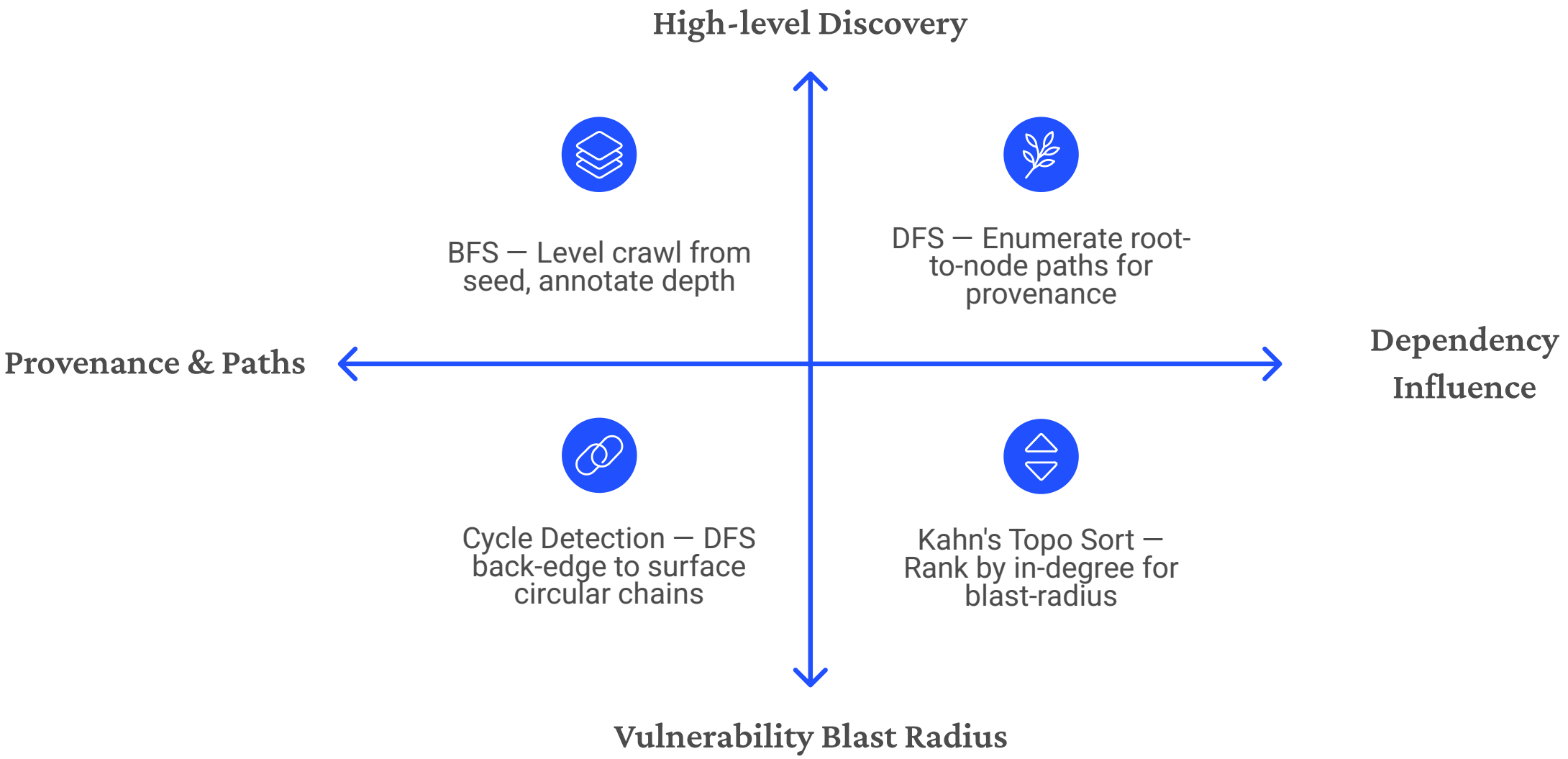
Real Attack: event-stream (2018)

Malicious package `flatmap-stream` was injected as a dependency of `event-stream`, targeting Bitcoin wallets. The compromised package had **8 million weekly downloads** before detection.

⚠ **The only way to find it:** materialize the full dependency graph and traverse it.

Treat npm as a Directed Graph

Four classical graph algorithms, hand-implemented in Python — no NetworkX. Every discovered package is cross-checked against the **OSV vulnerability database** (same source as GitHub Dependabot).

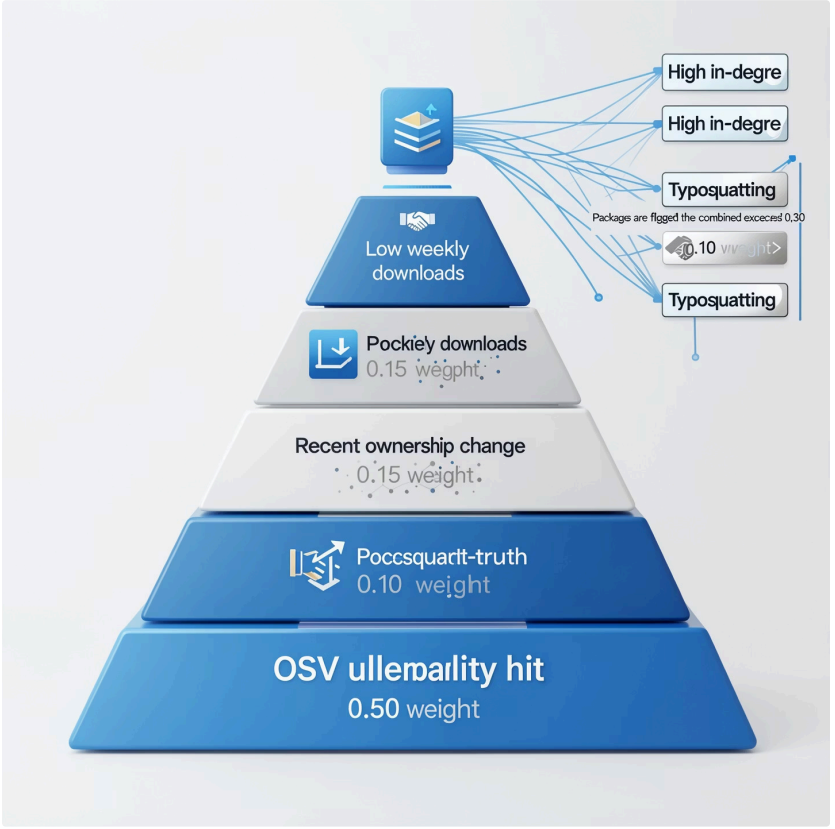


Each algorithm exposes a distinct attack surface: depth anomalies, provenance chains, forbidden cycles, and high-impact nodes.

Suspicion Scoring: 5 Weighted Signals

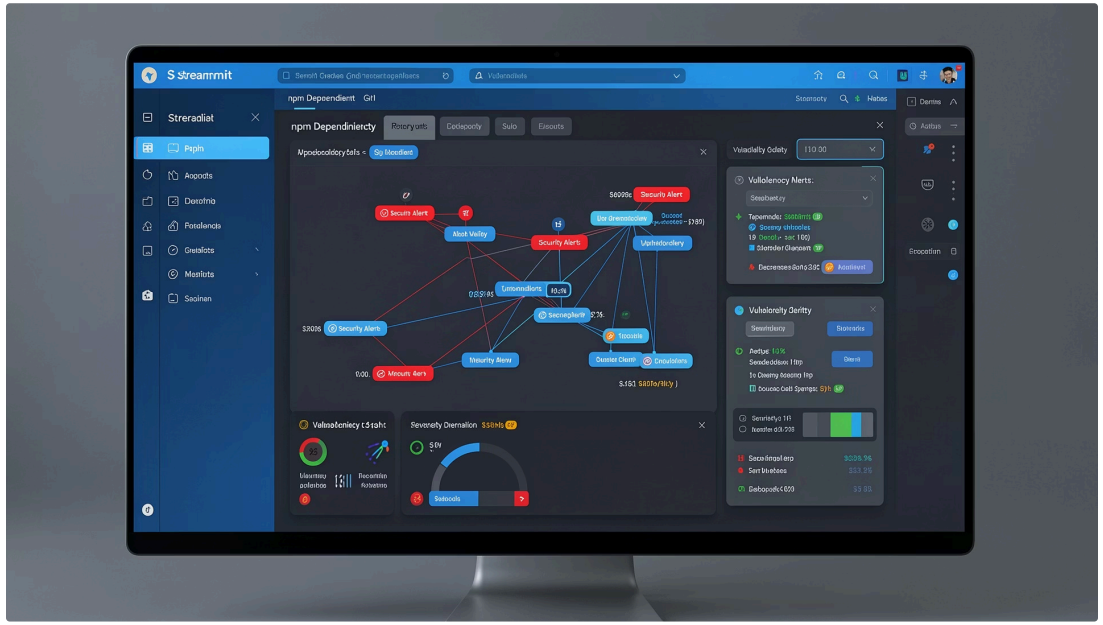
Each package receives a **0–1 suspicion score** from five weighted signals. Flagged when score exceeds threshold (default **0.30**, tunable).

Signal	Weight	What it captures
OSV vulnerability hit	**0.50**	Hard ground-truth label
Low weekly downloads	0.15	Near-zero downloads under a popular parent
Recent ownership change	0.15	Maintainer transfer before flag
Typosquatting	0.10	Levenshtein ⩽ 2 to a popular package name
High in-degree	0.10	Many packages depend on this node



Live Demo: Streamlit UI

Real npm data, real vulnerabilities — graph stats, dependency paths, and signal breakdowns all visible in the dashboard.



🚨 flatmap-stream → CRITICAL

Suspicion score: **0.50** (OSV hits only). OSV IDs: `GHSA-mh6f-8j2x-4483`, `MAL-2025-20690`. Malicious across all versions — no fix exists.

✅ express → 47 packages, 84 edges

Zero flagged. Top blast-radius node: `debug` (5 dependents), followed by `parseurl` and `statuses`.

Results & Key Takeaways

BFS Crawler

Live npm registry with on-disk caching — fully offline after first run

4 Algorithms

BFS, DFS, cycle detection, Kahn's toposort — all hand-implemented in Python

5-Signal Scorer

Tunable threshold, 37/37 tests passing, clickable OSV links in UI

Streamlit Dashboard

Real-time flagging, graph stats, dependency paths, and signal breakdowns

> **Key insight:** Graph traversal exposes attack surface that linear inspection cannot — the same technique used by production tools like Dependabot and Socket.dev, implemented from scratch using algorithms from class.